

Making Room Before the Next Miss: Capped CLOCK with a Soft Free-Space Reserve

FICTIONAL DEMO — specification and authored numerical fixtures only. No implementation, simulation, trace replay, experiment, or deployment has been performed.

Abstract

A large object may require a cache to inspect many small residents before enough bytes become available. Capping that scan limits reference examinations but can reject admission. This design note considers moving some eviction work to the end of earlier requests: 48 foreground examinations and 16 maintenance examinations share a 64-examination allowance, with maintenance seeking a soft free-space reserve. A simple 64-examination cap already supplies the same bound; the question is whether preparing space justifies discarding residents earlier. In fictional MixedBurst values, an 8 MiB reserve raises request hits from 63.50% to 67.00% while average examinations rise from 12 to 14. SteadySmall instead loses hits and fetches more bytes. The specification makes this tradeoff explicit; the fixtures establish neither performance nor an optimal reserve.

1. What remains after scanning is capped?

Consider an image-service cache receiving a 1 MiB object when its circular resident list contains many small objects. A recently accessed object gets a second chance: the scanner clears its reference bit and moves on. Even if every bit is clear, a full cache containing only 4 KiB residents needs 256 distinct victims to free 1 MiB. A 64-examination cap cannot admit that object without space prepared earlier. Set bits add inspections that release nothing; uncapped CLOCK can continue until enough victims are found.

The local Cap64 policy answers the immediate work-bound question without a controller or frequency estimator. It examines at most 64 references and bypasses insertion when insufficient space is found. Our reserve variants ask a narrower question: can earlier requests prepare free bytes for later admissions while retaining the same examination ceiling? Their contribution here is a specified division of the allowance, together with the admission and accounting rules needed to interpret it. These are design decisions, not completed engineering or a claim that free-space reserves are original.

Admission and filtering already have substantial precedent. TinyLFU compares a candidate and an eviction candidate using approximate recent frequencies [1]. S3-FIFO uses small, main, and payload-free ghost queues to filter low-reuse objects [2]. AdaptSize tunes size-dependent admission probabilities using a Markov cache model, targeting object-hit ratio in a CDN hot-object cache [3]. The reserve here is empty payload capacity, not S3-FIFO's small queue. None of these published systems was implemented for this DEMO, so their measured results cannot serve as its baselines or evidence.

Same scan ceiling, different allocation

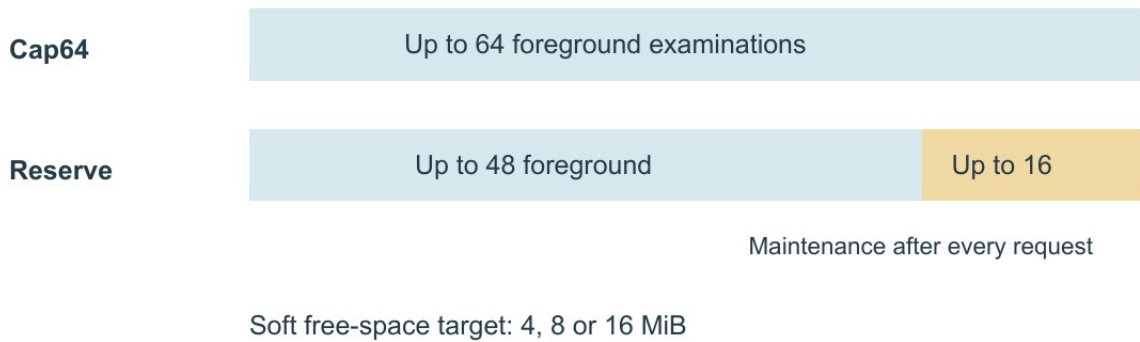


Figure 1. Allocation of the reference-examination allowance. Bar length represents the specified maximum, not elapsed time or observed work. The reserve variants use separate limits of 48 foreground and 16 maintenance examinations; maintenance seeks a soft free-space target and may stop earlier or leave it unmet.

2. Preparing bytes without hiding work

All five policies share a 64 MiB payload capacity, an exact used-byte counter, an index, and a circular list with one reference bit per resident. Requests execute serially; immutable keys keep their sizes and contents. There is no expiration, deletion, concurrency, failure handling, crash recovery, cancellation, or tenant isolation. Metadata, temporary victim pointers, response buffers, and allocation overhead are outside the capacity, which therefore does not bound server memory.

On a hit, the cache sets the reference bit. On a miss, it fetches the entire payload and inserts immediately if space suffices. Otherwise, the foreground scanner inspects the object at the hand, then advances the hand; it clears a set reference bit or records a clear-bit object as a tentative victim. Every inspection counts, including revisits, but a selected object's bytes count only once. Counting the same victim twice could otherwise authorize an insertion without enough space.

Victims remain resident until free plus selected bytes cover the candidate. A successful selection commits their removal and then inserts the object; an unsuccessful selection discards the victim list and bypasses insertion. Bit clearing and hand movement persist on bypass. Thus Cap64 does not discard useful payloads merely to discover that admission is impossible. Bypass still returns the fetched object to its requester.

Committing removal must move the hand to a surviving object if its target was selected; an empty ring has a null hand. New objects receive bit 1. In a nonempty ring, they enter immediately before the hand, at the end of the current cycle; insertion into an empty ring initializes the hand to the new object. Exact accounting must maintain used payload between zero and capacity after every committed operation. Bounded pointer storage suffices for capped selection, although duplicate-membership checks have costs outside the examination metric. These are specification requirements, not invariants verified by the aggregate table.

CLOCK leaves foreground examinations uncapped and performs no maintenance. Cap64 permits 64 foreground examinations. Reserve4, Reserve8, and Reserve16 instead permit 48 foreground plus 16 maintenance examinations, targeting 4, 8, and 16 MiB free, respectively. After every request, including a hit or bypass, maintenance operates while free bytes are below the target. It clears a set reference bit and advances; for a clear bit, it advances and immediately evicts the object. It stops when its allowance ends, the ring empties, or the target is reached. Its evictions are not rolled back.

The target is soft: insertion may consume the reserve, referenced residents may prevent restoration, and the last eviction may overshoot it. Maintenance can even evict the object just served or inserted. Separate response ownership makes this safe in the serial specification but cannot prevent lost reuse. There is no background worker or uncharged catch-up pass: each reserve request has at most $48 + 16 = 64$

examinations. Unlinking, hashing, membership checks, hit-bit writes, copies, allocation, and origin service remain outside that bound. It is not a response-time or total-CPU guarantee. Oversize objects bypass immediately; none occurs in the fixtures.

3. What the fictional comparisons contain

Table 1 retains all 20 policy–scenario combinations. Each row assigns one million requests, including an empty-cache start, across 4 KiB, 64 KiB, and 1 MiB payloads. SteadySmall denotes stable small-object reuse; MixedBurst adds large-object bursts; HotShift changes popular keys; LargeReuse emphasizes large-object reuse. These descriptions and matching size-class counts do not provide identical ordered traces, or even certify realizable cache trajectories.

All rows are hand-authored and have zero repetitions. Hits, examinations, bypasses, and maxima are fixtures; hit ratios and byte totals are arithmetic derivatives. Origin bytes equal the sum of missed requests times their class sizes, counting repeated fetches. Integer byte totals are authoritative; MiB uses 2^{20} bytes. A budget bypass is a miss rejected because the foreground allowance expires. No latency samples, hardware, trial variance, or statistical significance accompany these values.

Table 1. FICTIONAL DEMO, not measured: request hits, origin fetches, admission bypasses, and charged examinations. Read Cap64 against the reserve variants within each scenario; CLOCK shows the uncapped reference. All policies have the same 64 MiB capacity.

Scenario	Policy	Hits (%)	Fetches (MiB)	Bypasses	Mean exams	Max exams
SteadySmall	CLOCK	92.50	11718.7500	0	0.85	2048
SteadySmall	Cap64	92.25	12285.1562	2500	0.82	64
SteadySmall	Reserve4	91.45	12550.7812	500	1.40	64
SteadySmall	Reserve8	90.10	13453.1250	250	1.80	64
SteadySmall	Reserve16	86.80	15398.4375	120	2.40	64
MixedBurst	CLOCK	63.00	87578.1250	0	15.00	8192
MixedBurst	Cap64	63.50	97226.5625	12000	12.00	64
MixedBurst	Reserve4	65.20	89835.9375	7000	13.00	64
MixedBurst	Reserve8	67.00	81562.5000	4000	14.00	64
MixedBurst	Reserve16	66.20	76789.0625	2000	17.00	64
HotShift	CLOCK	59.50	95625.0000	0	12.00	6144
HotShift	Cap64	58.50	100761.7188	16000	10.00	64
HotShift	Reserve4	59.50	95566.4062	10000	10.50	64
HotShift	Reserve8	60.60	90250.0000	6000	12.00	64
HotShift	Reserve16	59.30	86667.9688	3000	14.00	64
LargeReuse	CLOCK	82.00	74531.2500	0	3.00	4096
LargeReuse	Cap64	80.20	94582.0312	20000	2.70	64
LargeReuse	Reserve4	80.30	92585.9375	16000	3.20	64
LargeReuse	Reserve8	79.80	89851.5625	14000	4.00	64
LargeReuse	Reserve16	78.50	80527.3438	7000	5.50	64

4. Admission opportunity has a residency cost

The Cap64-reserve comparison changes both the foreground allowance (64 versus 48 examinations) and maintenance (none versus up to 16), together with the reserve target. It therefore compares the complete budget split and reserve policy, not an isolated maintenance effect.

MixedBurst illustrates the intended opportunity. Cap64 to Reserve8 changes hits from 635,000 to 670,000, origin fetches from 97,226.5625 to 81,562.5000 MiB, and budget bypasses from 12,000 to 4,000.

Examinations rise from 12 million to 14 million. CLOCK's assigned maximum is 8,192, compared with 64 for both capped variants; this comparison cannot favor Reserve8 over Cap64 on the bound itself.

SteadySmall supplies the counterexample: Cap64 to Reserve8 lowers request hits from 92.25% to 90.10%, raises origin fetches from 12,285.1562 to 13,453.1250 MiB, and increases mean examinations from 0.82 to 1.80. HotShift instead assigns higher hits, 58.50% to 60.60%, with examinations increasing from 10 to 12 per request. LargeReuse separates the objectives: Cap64 to Reserve8 lowers request hits from 80.20% to 79.80% yet reduces origin fetches from 94,582.0312 to 89,851.5625 MiB, while mean examinations rise from 2.70 to 4.00. CLOCK retains the highest LargeReuse hit ratio, 82.00%, and the lowest origin fetches, 74,531.2500 MiB, at an uncapped assigned maximum of 4,096.

Changing the target does not yield a uniform winner. MixedBurst hit ratios for Reserve4, Reserve8, and Reserve16 are 65.20%, 67.00%, and 66.20%, although Reserve16 has the lowest origin-fetch total of the three. SteadySmall's corresponding ratios decline from 91.45% through 90.10% to 86.80%. Increasing reserved space can sacrifice useful residents and add scanning; the authored values illustrate that possibility without measuring its cause or identifying an optimum.

5. Scope and conclusion

The useful distinction is between stopping a long scan and preparing space before the next miss. Cap64 already does the former. A soft reserve specifies the latter while charging maintenance to the same request ceiling, at the expense of resident contents and additional average examinations. The fixtures motivate comparing those costs with both request hits and origin bytes; they cannot decide whether the policy should be deployed.

An implementation and executable traces are still needed to check state evolution, memory accounting, and actual work. Measurements under a stated service objective would be required for throughput, latency, or cost claims and fair comparisons with published policies. Prior-art coverage of bounded scanning and reserve scheduling remains incomplete. The present outcome is a concrete hypothetical design and a conditional question for evaluation, not established novelty or empirical cache improvement.

References

- [1] Gil Einziger, Roy Friedman, and Ben Manes. *TinyLFU: A Highly Efficient Cache Admission Policy*. arXiv:1512.00727. [Author-uploaded paper](#). Corresponds to S1 in the supplied sources.md.
- [2] Juncheng Yang, Yazhuo Zhang, Ziyue Qiu, Yao Yue, and K. V. Rashmi. *FIFO Queues are All You Need for Cache Eviction*. SOSP 2023. DOI: 10.1145/3600006.3613147. [Author-hosted paper](#). Corresponds to S2 in the supplied sources.md.
- [3] Daniel S. Berger, Ramesh K. Sitaraman, and Mor Harchol-Balter. *AdaptSize: Orchestrating the Hot Object Memory Cache in a Content Delivery Network*. NSDI 2017, pp. 483–498. [Conference paper](#). Corresponds to S3 in the supplied sources.md.